

**VNUHCM, UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**



**SUBMISSION OF SELF-PRACTICE
EVIDENCE FOR PROGRAMMING**

PART 3

SUBJECT: FUNDAMENTALS OF PROGRAMMING

Instructors: Bùi Duy Đăng

Nguyễn Thanh Tình

Class: 25C01

NO	STUDENT ID	FULL NAME
1	25127113	Võ Thiện Trí Nhân

Ho Chi Minh City - 12/2025

1 Introduction

Greetings! My name is Võ Thiện Trí Nhân, a student studying Information Technology at Ho Chi Minh City University of Science. I wrote this report to monitor my self-practice progress in learning programming.

My job is to solve problems on online platforms such as LeetCode or HackerRank (Codeforces is excluded from the list) and record solved exercises in this report. The online platform I mainly used to solve problems is Leetcode, one of the most well-known websites for practicing programming and mindset for interviews. The reason I chose this website is because of familiarity; I used to find exercises to practice on this website. To be honest, I couldn't solve most of them.

This is the end of the introduction. I hope this report will be accepted. Hope the readers have a good day.

2 Problems

2.1 Problem 1

- Name: Remove Element
- Link: [Leetcode - Remove Element](#)
- Level: Easy

2.1.1 Solution

```
#include <iostream>
#include <vector>

using namespace std;

class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int i = 0;
        while (i < nums.size()) {
            if (nums[i] == val) {
                nums.erase(nums.begin() + i);
                i--;
            }
            i++;
        }
        return nums.size();
    }
};
```

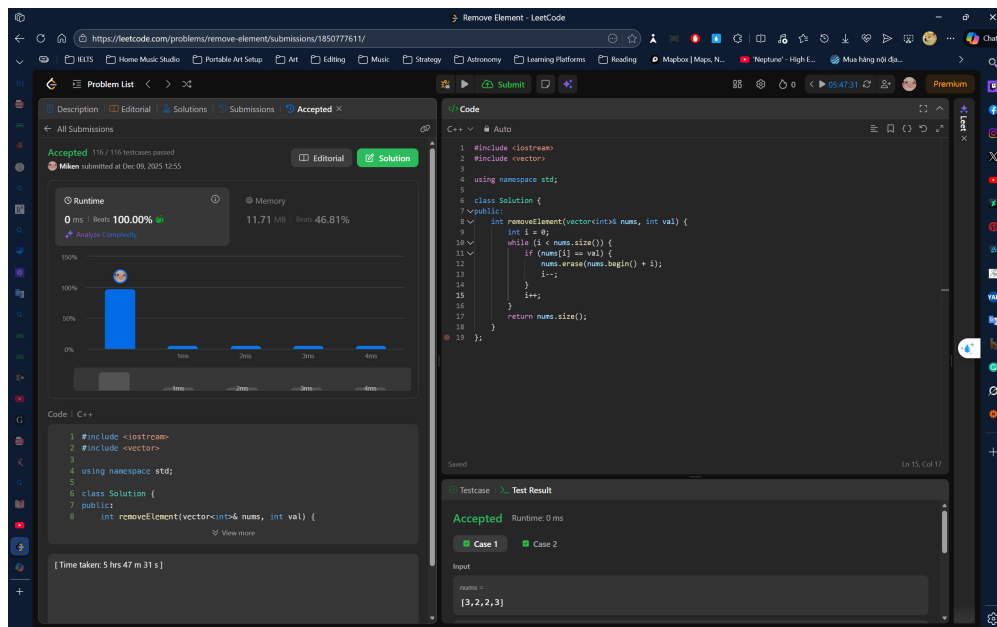
2.1.2 Description

The problem tells us to remove all the elements that equal to the chosen value.

1. Loop through the array
 - Initialize a variable 'i' which acts as a counter.
2. Check if the current element is equal to the chosen value
 - We use while loop for a much easier control.
 - We stop checking if the counter is equal to the number of elements of the list because the index starts at 0.
 - If yes. Erase the element and slide the counter backward (To double check the position to ensure all the same elements standing nearby are erased).
 - If no. Slide the counter forward and continue checking.
3. Return the array

2.1.3 Test cases

1. Case 1: nums = [4, 4, 5, 4], val = 4 $\implies$ Output: [5]
2. Case 2: nums = [10, 12, 13, 5], val = 5 $\implies$ Output: [10, 12, 13]
3. Case 3: nums = [50, 33, 33, 44], val = 33 $\implies$ Output: [50, 44]



Hình 1: Problem 1: Remove Element

2.2 Problem 2

- Name: Merge Sorted Array
- [Leetcode - Merge Sorted Array](#)
- Level: Easy

2.2.1 Solution

```
#include <algorithm>

class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        int total = m + n;
        for (int i = m, j = 0; i < total, j < n; i++, j++) {
            nums1[i] = nums2[j];
        }

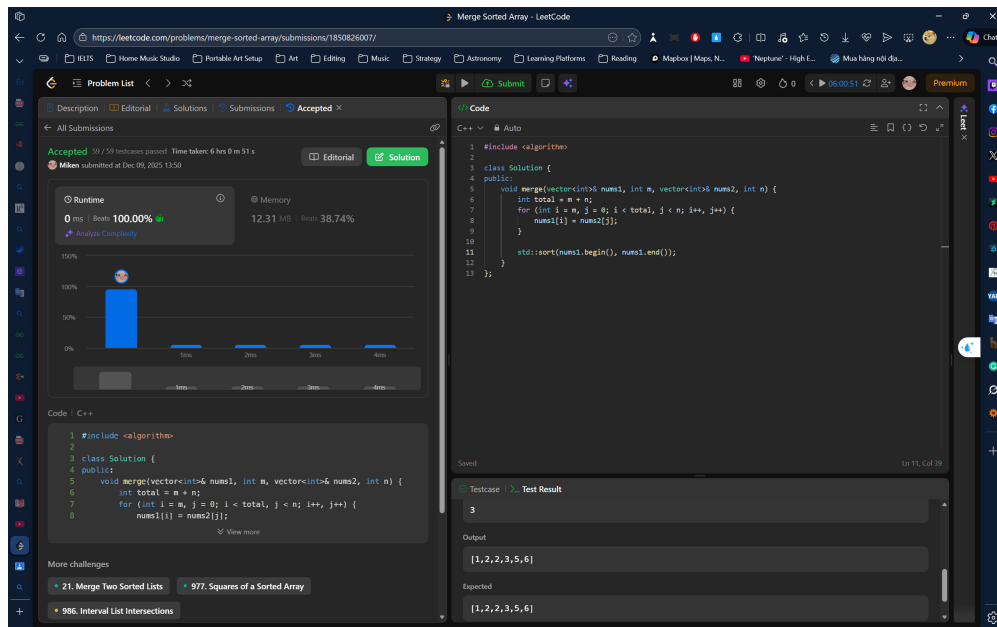
        std::sort(nums1.begin(), nums1.end());
    }
};
```

2.2.2 Description

1. The number of elements of the new list is equal to the sum of the numbers of elements of two given arrays.
 - We initialize a variable storing the sum of the numbers of elements of two given arrays.
2. We use for loop to loop through the first array.
 - We will start adding the new elements at the m-th position (m is the number of elements of the first array).
3. Sort the new array after adding.

2.2.3 Test cases

1. Case 1: $\text{nums1} = [1, 4, 7, 0, 0, 0, 0]$, $m = 3$, $\text{nums2} = [2, 3, 5, 6]$, $n = 4 \implies$ Output: $[1, 2, 3, 4, 5, 6, 7]$
2. Case 2: $\text{nums1} = [10, 20, 30, 0, 0, 0]$, $m = 3$, $\text{nums2} = [1, 2, 3]$, $n = 3 \implies$ Output: $[1, 2, 3, 10, 20, 30]$
3. Case 3: $\text{nums1} = [5, 6, 7]$, $m = 3$, $\text{nums2} = []$, $n = 0 \implies$ Output: $[5, 6, 7]$



Hình 2: Problem 2: Merge Sorted Array

2.3 Problem 3

- Name: Length of Last Word
- [Leetcode - Length of Last Word](#)
- Level: Easy

2.3.1 Solution

```
#include <cctype>
#include <iostream>

class Solution {
public:
    int lengthOfLastWord(string s) {
        int i = s.length() - 1;

        while (i >= 0) {
            if (!isspace(s[i])) {
                break;
            } else {
                s.erase(s.begin() + i);
            }
            i--;
        }
    }
}
```

```

        int result = 0;

        for (int j = s.length() - 1; j >= 0; j--) {
            if (isspace(s[j])) {
                break;
            } else {
                result++;
            }
        }

        return result;
    }
};

```

2.3.2 Description

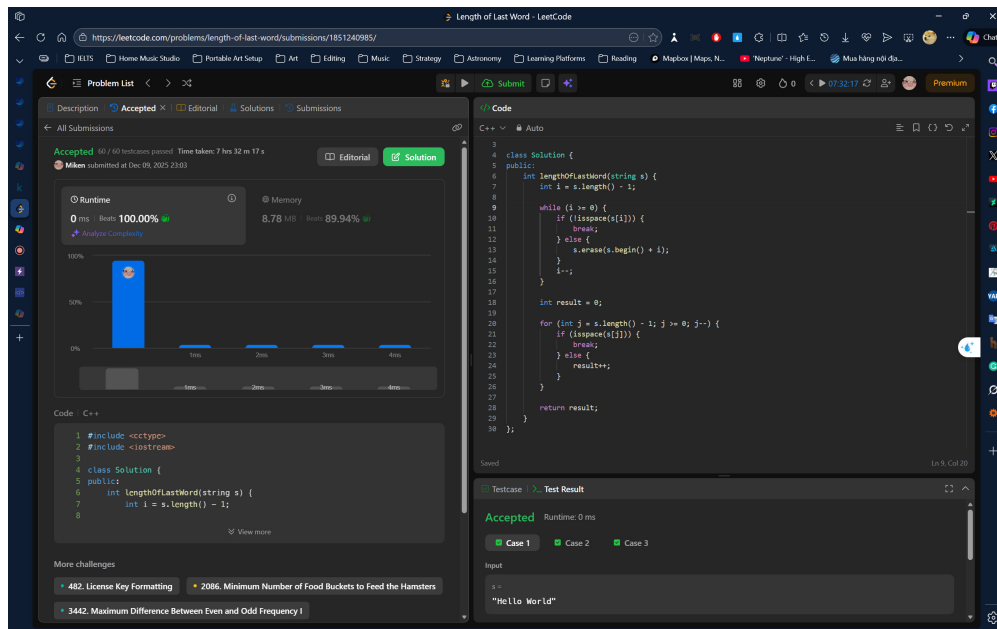
1. To count the length of last word, we just need to count backwards (starts at the end of the character array).
2. Erases all the white space after the last word.
 - Starts at n-1 (n is the length of the character array).
 - Check if the current element is a space.
 - If yes. Erase that element.
 - If no. Break.
 - Start counting from (n-1)-th position. Initialize a variable storing the result.
 - If the element is a character, add one to the variable.
 - If the element is a space, break (Words are separated by spaces).

2.3.3 Test cases

1. Case 1: s = "Hello HCMUS " $\implies$ Output: 5
2. Case 2: s = "Ho Chi Minh City University of Science " $\implies$ Output: 7
3. Case 3: s = "I love learning Information Technology" $\implies$ Output: 10

2.4 Problem 4

- Name: Merge Two Sorted Lists
- [Leetcode - Merge Two Sorted Lists](#)
- Level: Easy



Hình 3: Problem 3: Length of Last Word

2.4.1 Solution

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */

class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode* head = list1;
        ListNode* head2 = list2;
        ListNode* temp = list1;
        ListNode* temp2 = list2;

        if (temp == nullptr && temp2 == nullptr) {
            return head;
        }
        else if (temp == nullptr && temp2 != nullptr) {
            int n = 0, m = 0;
```

```

while (temp2->next != nullptr) {
    temp2 = temp2->next;
    m++;
}

temp2->next = list1;

bool swapped = false;
ListNode* previous = head2;
ListNode* next = head2->next;

for (int i = 0; i < n + m; i++) {
    swapped = false;
    previous = head2;
    next = head2->next;
    for (int j = 0; j < (n + m) - i - 1; j++) {
        if (!next)
            break;

        if (previous->val > next->val) {
            int temp2 = previous->val;
            previous->val = next->val;
            next->val = temp2;
            swapped = true;
        }

        previous = previous->next;
        next = next->next;
    }

    if (!swapped) {
        break;
    }
}

return head2;
}
else if (temp != nullptr && temp2 == nullptr) {
    int n = 0, m = 0;

    while (temp->next != nullptr) {

```



```

        temp = temp->next;
        n++;
    }

    temp->next = list2;

    bool swapped = false;
    ListNode* previous = head;
    ListNode* next = head->next;

    for (int i = 0; i < n + m; i++) {
        swapped = false;
        previous = head;
        next = head->next;
        for (int j = 0; j < (n + m) - i - 1; j++) {
            if (!next)
                break;

            if (previous->val > next->val) {
                int temp2 = previous->val;
                previous->val = next->val;
                next->val = temp2;
                swapped = true;
            }

            previous = previous->next;
            next = next->next;
        }

        if (!swapped) {
            break;
        }
    }

    return head;
}
else {
    int n = 0, m = 0;
    while (temp->next != nullptr) {
        temp = temp->next;
        n++;
    }
}

```

```

n++;

temp->next = list2;

while (temp != nullptr) {
    temp = temp->next;
    m++;
}
m++;

bool swapped = false;
ListNode* previous = head;
ListNode* next = head->next;

for (int i = 0; i < n + m; i++) {
    swapped = false;
    previous = head;
    next = head->next;
    for (int j = 0; j < (n + m) - i - 1; j++) {
        if (previous->val > next->val) {
            int temp2 = previous->val;
            previous->val = next->val;
            next->val = temp2;
            swapped = true;
        }

        previous = previous->next;
        next = next->next;

        if (!next)
            break;
    }

    if (!swapped) {
        break;
    }
}

return head;
}
};

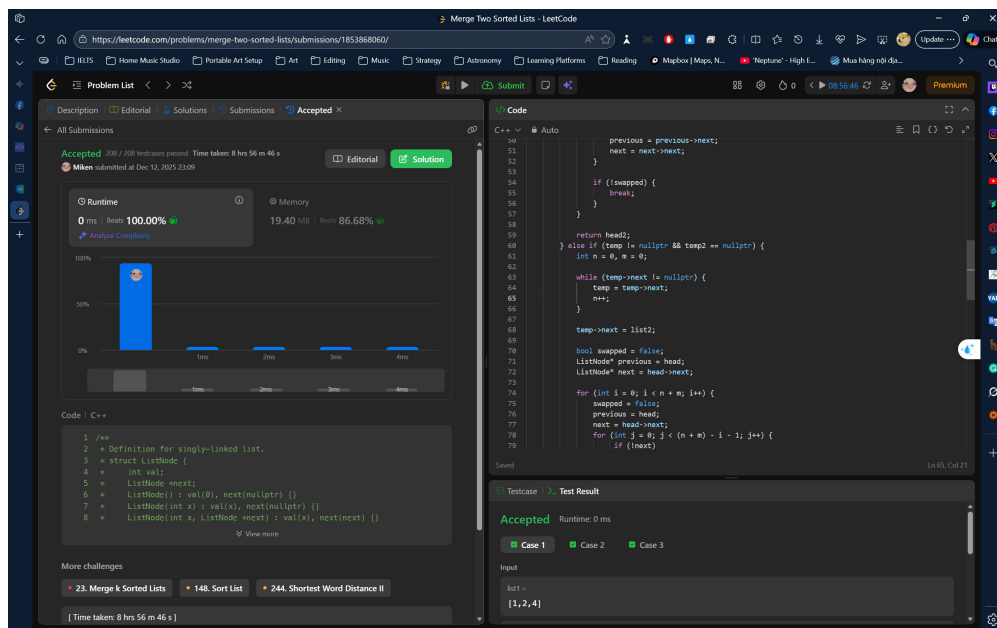
```

2.4.2 Description

1. We need to link the two sorted list first.
 - Starts at the first node of the first linked list.
 - We use another pointer to find the last node of the first linked list.
 - Instead the last node's pointer pointing to NULL, we set it points to the first node of the second linked list.
2. Use Bubble Sort algorithm to sort the merge list.
 - We initialize two pointers, one pointing to the first node of the list, another one pointing to the second node of the list.
 - Compare the nodes' value where the pointers are pointing to.
 - Initialize a bool variable to check if the elements are swapped.
 - If the value of the left node is greater than the value of the right node, we swap the values.
 - Else we slide the pointers forward and continue checking.
 - When the bool variable is completely false (which means the elements are fully sorted), break out the loop.
3. Return the pointer pointing to the first node of the merged linked list.

2.4.3 Test cases

1. Case 1: list1 = [3,13,44], list2 = [2,10,55] $\Rightarrow$ Output: [2,3,10,13,44,55]
2. Case 2: list1 = [], list2 = [0, 1, 2] $\Rightarrow$ Output: [0,1,2]
3. Case 3: list1 = [20, 49], list2 = [0, 50] $\Rightarrow$ Output: [0, 20, 49, 50]



Hình 4: Problem 4: Merge Two Sorted Lists